



Gisma University
of Applied Sciences

Assessment Submission Form

Student Number	Aditya Kumar GH1039641
Assessment Title	Gym Management System
Module Code	B100
Module Title	Python Programming WS0926
Module Tutor	Ramin Baghaei Mehr
Date Submitted	18.09.2026

Declaration of Authorship

I declare that all material in this assessment is my own work except where there is clear acknowledgement and appropriate reference to the work of others. I fully understand that the unacknowledged inclusion of another person's writings or ideas or works in this work may be considered plagiarism and that, should a formal investigation process confirms the allegation, I would be subject to the penalties associated with plagiarism, as per GISMA Business School, University of Applied Sciences' regulations for academic misconduct.

Signed.....Aditya..... Date18.09.2026.....

Table of Contents

Introduction.....	3
1.1 Project Description.....	3
1.2 Objectives	3
System Design	4
2.1 System Requirements.....	4
2.2 Entity-Relationship Diagram	4
Implementation	6
Results.....	15
Conclusion and Future Work.....	18
Appendix.....	20

Introduction

1.1 Project Description

The current research provides a design and implementation of a full-fledged relational database of the Hotel Reservation System. The database includes all vital features of the hotel business such as customer management, registration management, room management and payments.

The current system is completely built on the SQL (Dbeaver) and follows all the principles of a relational database through setting constraints, maintaining referential integrity with clear primary and foreign key relations.

1.2 Objectives

The primary objectives of this project were:

1. Design a normalized relational database
2. Implementation of the relational database
3. Adding records in the database
4. Develop a set of SQL queries

System Design

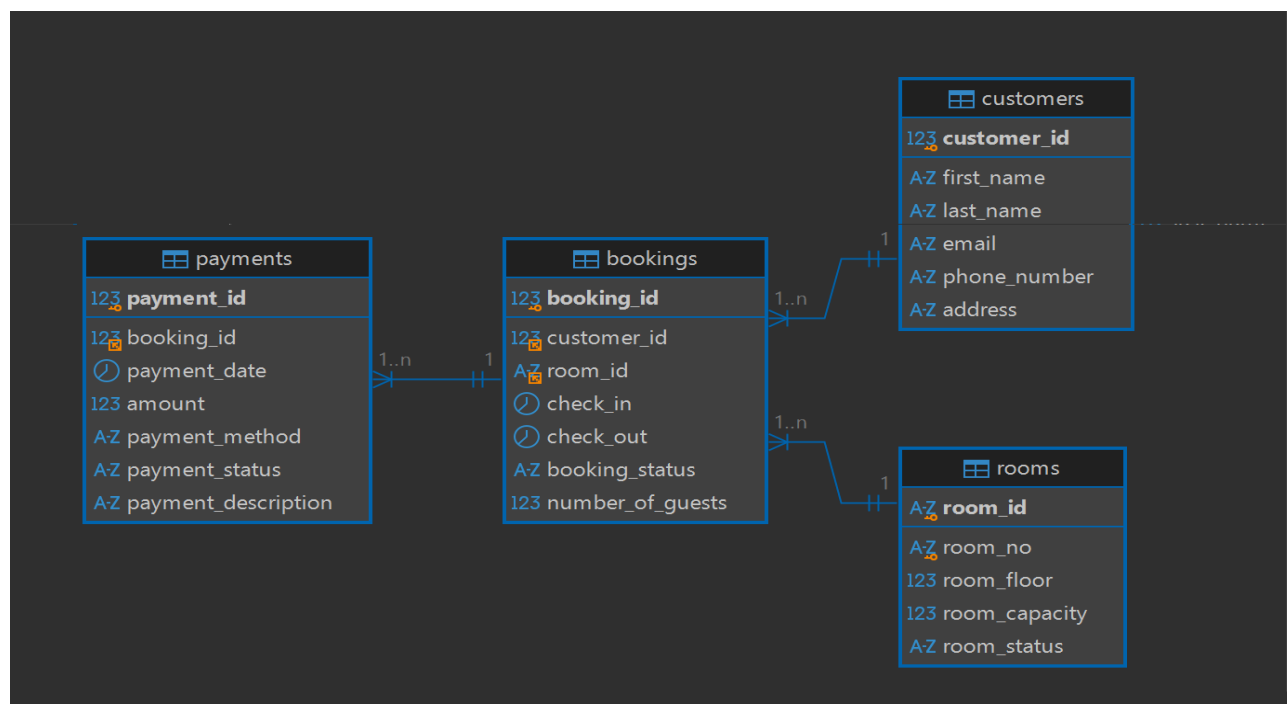
2.1 System Requirements

The hotel reservation system must support the following functional requirements:

1. Register and manage customer accounts with contact and address information.
2. Maintain a room inventory organized by floor, capacity and availability status.
3. Process bookings that associate a customer with a room for a specific date and time range.
4. Track booking status (Confirmed → Completed or Cancelled).
5. Automatically update room availability based on booking status and current time.
6. Record and validate payment transactions linked to bookings.
7. Track payment statuses (Paid, Pending, Refunded) and allow descriptive notes on payments.

2.2 Entity-Relationship Diagram

The ERD diagram identifies the following entities and relationships:



Implementation

1. Creating Database and selecting schema

```
-- Creating database
create database Hotel_reservation;

-- Select the database to edit
use Hotel_reservation;
```

2. Creating Customers and Rooms Table

```
-- Creating Customers Table
create table customers (
customer_id int auto_increment primary key,
first_name varchar(50) not null,
last_name varchar(50) not null,
email varchar(50) not null,
phone_number varchar(13) not null,
address varchar(50) not null
);

-- Creating Rooms Table
create table rooms (
room_id int auto_increment primary key,
room_no varchar(3) not null unique,
room_floor int not null,
room_capacity int not null,
room_status varchar(20) default 'Available',
check (room_capacity > 0),
check (room_status in ('Available', 'Occupied', 'Maintenance'))
);
```

3. Creating Bookings and Payment

(*used execute as sql script near bookings and payments table)

```
-- Creating Bookings Table (****run this as sql script)
create table bookings (
  booking_id int auto_increment primary key,
  customer_id int not null,
  room_id int not null,
  check_in datetime,
  check_out datetime,
  booking_status varchar(20) not null,
  number_of_guests int not null,

  foreign key (customer_id)
    references customers(customer_id),

  foreign key (room_id)
    references rooms(room_id),

  check (check_out > check_in),
  check (booking_status in ('Confirmed', 'Cancelled', 'Completed'))
);

-- Creating Payments Table (****run this as sql script)
create table payment (
  payment_id int not null auto_increment primary key,
  booking_id int not null,
  payment_date date not null,
  amount decimal(10,2) not null,
  payment_method varchar(20) not null,
  payment_status varchar(20) not null,
  payment_description varchar(30),

  foreign key (booking_id)
    references bookings(booking_id),

  check (amount > 0),
  check (payment_method in ('cash','card','bank transfer')),
  check (payment_status in ('paid','pending','refunded'))
);
```

4. Renaming and Changing datatype of the column

```
-- Renaming the table to 'Payments'
alter table payment
rename to payments;

-- Changing the datatype of room_id from 'int' to 'varchar'
alter table bookings
drop foreign key `2`;

alter table rooms
modify room_id varchar(5) not null;

alter table bookings
drop column room_id;

alter table bookings
add column room_id varchar(5) NOT NULL AFTER customer_id;

alter table bookings
add constraint `2`
foreign key (room_id)
references rooms(room_id);

-- To show how a table was created
show create table customers;
```

5. Inserting all the data in record

```
-- Inerting data in all the tables
insert into customers
(first_name, last_name, email, phone_number, address)
values
('John', 'Smith', 'john@gmail.com', '+491234567890', 'lichtstreet 12, Berlin, Germany'),
('Will', 'Smith', 'Will@gmail.com', '+351762096762', 'hellstreet 22, Berlin, Germany'),
('Tom', 'Hardy', 'tomi@hardy.com', '+491745556789', 'firetstreet 44, Essen, Germany'),
('Jimmy', 'Sandhu', 'Sandhu34@gmail.com', '+491234567890', 'waterstreet 32, Munich, Germany'),
('Ammy', 'James', 'Ammy12@gmail.com', '+491234567890', 'risestreet 25, Dresden, Germany');

insert into rooms
(room_id, room_no, room_floor, room_capacity)
values
('A101', 101, 1, 5),
('A102', 102, 1, 4),
('B201', 201, 2, 3),
('B202', 202, 2, 3),
('C301', 301, 3, 2),
('C302', 302, 3, 2),
('D401', 401, 4, 1),
('D402', 402, 4, 1),
('D403', 403, 4, 1);

insert into bookings
(customer_id, room_id, check_in, check_out, booking_status, number_of_guests)
values
(1, 'D401', current_timestamp, '2026-09-11 12:00:00', 'confirmed', 1),
(2, 'C302', current_timestamp, '2026-09-12 22:00:00', 'confirmed', 2),
(3, 'B202', '2026-09-09 15:35:45', '2026-09-13 01:30:00', 'confirmed', 3),
(4, 'A101', '2026-09-01 12:22:32', '2026-09-03 18:22:32', 'completed', 5),
(5, 'A102', '2026-09-25 08:00:00', '2026-09-27 12:00:00', 'confirmed', 4);

insert into payments
(booking_id, payment_date, amount, payment_method, payment_status, payment_description)
values
(1, '2026-09-10', 60.00, 'Card', 'Paid', null),
(2, '2026-09-02', 150.00, 'Card', 'Paid', null),
(3, '2026-09-01', 400.00, 'Cash', 'Pending', null),
(4, '2026-09-03', 590.00, 'Card', 'Paid', "added 50 for broken things"),
(5, '2026-09-04', 490.00, 'Bank Transfer', 'Pending', null);
```

6. Basic sql queries

```
● -- display all record
  select * from customers;

● -- adding a record
  insert into customers
  (first_name, last_name, email, phone_number, address)
  values
  ('Lexi', 'singh', 'singhisking@gmail.com', '+491762345634', 'alexastreet 59, Berlin, Germany');

● -- update a record
  update customers
  set phone_number = '+491234569089'
  where customer_id = 3;

● -- deleting a record
  delete from customers
  where customer_id = '6';

● -- count total rooms
  select count(*) as total_rooms
  from rooms
  group by room_status;

● -- average revenue
  select avg(amount) as Average_revenue
  from payments
  where payment_status = 'Paid';

● -- room for 3 or more guests
  select * from rooms
  where room_capacity > 2;
```

7. Some examples of join queries

```
● -- rooms costing more than average
select
    r.room_no,
    r.room_id,
    p.amount,
    b.booking_id
from bookings b
join payments p
    on b.booking_id = p.booking_id
join rooms r
    on b.room_id = r.room_id
where p.amount > (
    select avg(amount)
    from payments);

● -- join query (It gives all the booking information along with the customer and room information)
select
    c.first_name,
    c.last_name,
    r.room_id,
    b.check_in,
    b.check_out,
    b.booking_status
from bookings b
join customers c
    on b.customer_id = c.customer_id
left join rooms r
    on b.room_id = r.room_id;
```

```
● -- all customers and their bookings
select
    c.customer_id,
    c.first_name,
    c.last_name,
    b.booking_id,
    b.room_id,
    b.check_in,
    b.check_out
from customers c
left join bookings b
    on c.customer_id = b.customer_id

union

● select
    c.customer_id,
    c.first_name,
    c.last_name,
    b.booking_id,
    b.room_id,
    b.check_in,
    b.check_out
from customers c
right join bookings b
    on c.customer_id = b.customer_id;
```

8. Procedure Example

```
-- a stored procedure to get booking information as per customer_id

delimiter //

● create procedure Get_customer_bookings(in customerid int)
begin
    select
        b.booking_id,
        c.first_name,
        c.last_name,
        r.room_no,
        b.check_in,
        b.check_out,
        b.booking_status,
        b.number_of_guests
    from bookings b
    join customers c
        on b.customer_id = c.customer_id
    join rooms r
        on b.room_id = r.room_id
    where b.customer_id = customerid;
end

delimiter //

call get_customer_bookings(3);
```

9. Event to update the status of the room

```
-- Event to update the status of the room

set global event_scheduler = on;

delimiter //

-- create event update_booking_room_status
on schedule every 1 minute
do
begin

-- update bookings
set booking_status = 'Completed'
where booking_status = 'Confirmed'
and current_timestamp >= check_out;

-- update bookings
set booking_status = 'Confirmed'
where booking_status not in ('completed', 'Cancelled')
and current_timestamp >= check_in
and current_timestamp < check_out;

-- update rooms r
join bookings b
on r.room_id = b.room_id
set r.room_status = 'Occupied'
where b.booking_status = 'confirmed'
and current_timestamp >= b.check_in
and current_timestamp < b.check_out;

-- update rooms r
join bookings b
on r.room_id = b.room_id
set r.room_status = 'Available'
where b.booking_status in ('Completed', 'Cancelled')
or current_timestamp >= b.check_out;

end //

delimiter;
```

10. Transaction Example

```
-- Transaction example
| start transaction;

● insert into customers
  (first_name, last_name, email, phone_number, address)
  values
  ('Tom', 'raj', 'rajtom@gmail.com', '+491235670989', 'warstreet 43, Berlin, Germany');

  set @customer_id = last_insert_id();

● insert into bookings
  (customer_id, room_id, check_in, check_out, booking_status, number_of_guests)
  values
  (last_insert_id(), 'C302', '2026-09-15 12:00:00', '2026-09-17 11:00:00', 'Confirmed', '2');

● set @booking_id = last_insert_id();
  insert into payments
  (booking_id, payment_date, amount, payment_method, payment_status)
  values
  (@booking_id, '2026-09-25', 240, 'card', 'pending');

  commit;
```

Results

1. Output of display all record

	123 customer_id (1)	A-Z first_name (2)	A-Z last_name (3)	A-Z email (4)	A-Z phone_number	A-Z address (6)
1	1	John	Smith	john@gmail.com	+491234567890	lichtstreet 12, Berlin, Germany
2	2	Will	Smith	Will@gmail.com	+351762096762	hellstreet 22, Berlin, Germany
3	3	Tom	Hardy	tomi@hardy.com	+491234569089	firestreet 44, Essen, Germany
4	4	Jimmy	Sandhu	Sandhu34@gmail.com	+491234567890	waterstreet 32, Munich, Germany
5	5	Ammy	James	Ammy12@gmail.com	+491234567890	risestreet 25, Dresden, Germany
6	7	Tom	raj	rajtom@gmail.com	+491235670989	warstreet 43, Berlin, Germany

2. Count total rooms

	123 total_rooms (1)
1	9

3. Calculate average revenue

	123 Average_revenue (1)
1	266.666667

4. Rooms for 3 or more guests

	A-Z room_id (1)	A-Z room_no (2)	123 room_floor (3)	123 room_capacity (4)	A-Z room_status (5)
1	A101	101	1	5	Available
2	A102	102	1	4	Available
3	B201	201	2	3	Available
4	B202	202	2	3	Available

5. Rooms costing more than average

	A-Z room_no (1)	A-Z room_id (2)	123 amount (3)	123 booking_id (4)
1	202	B202	400	3
2	101	A101	590	4
3	102	A102	490	5

6. Details of booking along with room and customer information

	AZ first_name (1)	AZ last_name (2)	AZ room_id (3)	check_in (4)	check_out (5)	AZ booking_status (6)
1	John	Smith	D401	2026-09-10 22:00:00.000	2026-09-11 12:00:00.000	Completed
2	Will	Smith	C302	2026-09-10 22:00:00.000	2026-09-12 22:00:00.000	Completed
3	Tom	Hardy	B202	2026-09-09 15:35:45.000	2026-09-13 01:30:00.000	Completed
4	Jimmy	Sandhu	A101	2026-09-01 12:22:32.000	2026-09-03 18:22:32.000	completed
5	Ammy	James	A102	2026-09-25 08:00:00.000	2026-09-27 12:00:00.000	confirmed
6	Tom	raj	C302	2026-09-15 12:00:00.000	2026-09-17 11:00:00.000	Completed

7. All customers and their bookings

	I23 customer_id (1)	AZ first_name (2)	AZ last_name (3)	I23 booking_id (4)	AZ room_id (5)	check_in (6)	check_out (7)
1	1	John	Smith	1	D401	2026-09-10 22:00:00.000	2026-09-11 12:00:00.000
2	2	Will	Smith	2	C302	2026-09-10 22:00:00.000	2026-09-12 22:00:00.000
3	3	Tom	Hardy	3	B202	2026-09-09 15:35:45.000	2026-09-13 01:30:00.000
4	4	Jimmy	Sandhu	4	A101	2026-09-01 12:22:32.000	2026-09-03 18:22:32.000
5	5	Ammy	James	5	A102	2026-09-25 08:00:00.000	2026-09-27 12:00:00.000
6	7	Tom	raj	6	C302	2026-09-15 12:00:00.000	2026-09-17 11:00:00.000

8. Get the booking information of the customer by their customer id

	I23 booking_id (1)	AZ first_name (2)	AZ last_name (3)	AZ room_no (4)	check_in (5)	check_out (6)	AZ booking_status (7)	I23 number_of_guests (8)
1	3	Tom	Hardy	202	2026-09-09 15:35:45.000	2026-09-13 01:30:00.000	Completed	3

Challenges and Solutions

One of the difficulties which emerged during the design of the Gym Management System is associated with database design, establishment of inter-table relationships and implementation of SQL queries. Firstly, the problem was in understanding how different entities, such as customers, rooms, bookings and payments relate to each other. It was critical to define correctly primary keys and foreign keys. These problems were solved creation of an ER diagram which helped to get an overall picture of the SQL database and then assign primary keys for unique record identification and establish connections using foreign keys.

Programming mistakes also took place, including such syntax errors like rename command to change a name of a table from 'payment' to 'payments'. Besides, it was necessary to take into account the data type of each column at the very beginning of the database design, since changing the data type of a column used as a foreign key in other table requires dropping a foreign key constraint and making changes in related tables. For instance, the data type of 'room_id' was changed from int to varchar.

Data integrity was another difficulty that was linked to the need to preserve data consistency. The requirement is that payments should be connected to an existing customer and be recorded properly; this can be achieved by using a foreign key constraint.

All the above challenges helped to make the database more reliable and to gain a lot of useful experience in database designing, SQL query implementation, constraints setting, relationships creation, and queries testing.

Conclusion and Future Work

The Hotel Reservation System presents an example of a relational database creation and implementation using SQL involving primary keys, foreign keys, constraints and table to organize the data management. This system also provides an organized way to manage important hotel data such as customers, rooms, reservations and payments.

The project has provided the opportunity to see how the SQL can be used in different tasks, like inserting, retrieving, updating and deleting the data. Queries with joins, filters and aggregation provide the opportunity to receive the necessary information from the interconnected tables to perform routine tasks related to hotel reservations.

Moreover, working on the project helped to gain experience in database design, referential integrity, data consistency, and the use of SQL. Testing the database with the sample records has helped to identify possible errors and check if the queries provide expected results.

There are several directions for future improvement of the database. Firstly, A UI could be designed for the better management of the data. Privacy could also be a considering factor in this. It will be useful to create a user authentication and role-based authorization system to give different permissions to administrators, receptionists and other users.

Also, in the future, the following features can be implemented into the database: automatic check for room availability, online booking, automatic payment processing, cancellation, sending notifications to customers, booking confirmations by email, and creating reports about occupancy rate, income, room availability, and other statistics.

Thus, the current database provides the base for building the Hotel Reservation System, which in the future may become a complete hotel management system.

Appendix

Github Link :-

<https://github.com/avengeradi/Gisma/tree/main/B103%20Databases%20%26%20Big%20Data%20SS0626>

Video Link :- [SQL_Demonstration_Video.mp4](#)

https://gismauniversity-my.sharepoint.com/:v:/g/personal/aditya_kumar_gisma-student_com/IQDp8tDwx37jQrrT_7c1WuxcAW4YMMKqTcEAM3bmFw9xMQs?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAIoiJPbmVEcmI2ZUZvckJlc2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkiX0&e=OpwQhB